The counterexample belongs in the demo

Building an instrument around one measurable claim

We set out to make one behavior visible: in a small trained BDH, the fraction of active neurons in layer 2 falls roughly threefold when a novel word begins repeating. The interesting part is that the weights stay fixed throughout the sequence. There is no slider that tells the model how many neurons to activate. Our job is to expose that observation clearly enough that a learner can also find its limits.

This is an educational artifact for people who know activations, ReLU, and next-token prediction but may never have encountered BDH. We want them to inspect a result, change a condition, and recognize a counterexample. A compelling curve is only useful if its labels tell the reader what was measured. That principle determined the model variable, the controls, the comparison, and the documentation.

The experiment before the interface

The project record contains a useful false start. According to `probe.md`, an earlier external review trained a BDH on tiny Shakespeare and found little activity change on repeated text. That was the reviewer's experiment, not a new experiment conducted for this build. It tested a different training distribution from the synthetic protocol behind Figure 14 in the BDH paper. A negative result on language-trained repetition cannot by itself settle the paper's narrower synthetic claim.

Reading the primary source changes the experiment. Kosowski and colleagues train on cycles consisting of 13 fixed warm-up letters, followed by eight copies of a random eight-letter word. New cycles introduce new words. The first appearance supplies information; later appearances can be predicted from context. The paper measures sparse nonnegative activations in higher layers, with its own much larger model and repeated-cycle setting. The Dragon Hatchling, §6.4

Our local reproduction shrinks the measured neuron dimension from 65,536 to 1,024 and the embedding width from 256 to 32. It keeps four layers and uses four heads. The weights are shared across depth, giving 100,352 parameters. Training runs for 2,200 steps with AdamW, learning rate 0.003, batch size four, and sequence length 154. This makes the probe practical on CPU without pretending it is the published-scale model.

Measuring the right thing

A naming detail could have invalidated the whole instrument. The published activation is y = ReLU(D_y LN(a*)) (elementwise product) x. In the code, `y_sparse` names just one factor; `xy_sparse` names the product. We count strictly positive entries in the product and divide by 1,024. Four heads of 256 coordinates are flattened in head-major order, so the grid and the chart share the same denominator.

The equation supplies an exact structural explanation: with nonnegative factors, both must be positive for their product to be positive. It does not supply a complete behavioral explanation. In particular, it cannot prove that confidence makes one factor collapse. That distinction matters because the data later reject a simple uncertainty story. The parameter names map as D_x to `encoder`, D_y to `encoder_v`, and E to `decoder`.

On the preserved evaluation word `mmgtfhhe`, trained layer-2 activity averages 15.77% during the first word and 5.08% during subsequent copies. The ratio is 3.10x. These are our small-model phase means. The paper's Figure 14 reports 4.0–7.5% during memorization and approximately 2.5% during repetition at its own scale. Agreement in the direction of the effect is useful; identical percentages or an exact replication of

every condition would be a stronger claim than our evidence supports.

Controls that can disappoint us

The untrained model is the first check against telling a story from the algebra alone. It has the same architecture and a reproducible random initialization. At layer 2 its memorization-to-repeat ratio is approximately 1.04x, so the large trained drop is absent. That supports the conclusion that training matters for this observed behavior. It does not identify which learned feature is responsible, and one initialization is not a distribution over possible models.

Layer 0 provides a second boundary using the trained weights. Its phase means are approximately 12.84% and 13.91%, so there is no comparable drop. The curve still varies across tokens. Calling it perfectly flat would hide that detail, while describing an established inversion would imply statistical evidence we have not collected. The same matrices are applied at different depths to evolving representations; shared parameters do not mean identical activations.

The guide follows these controls deliberately: begin with trained layer 2, inspect layer 0, then restore layer 2 before selecting untrained weights. Otherwise the learner could see the right control value attached to the wrong layer. Direct interaction should end a guided step rather than compete with a delayed automatic selection. Exploration needs to remain the learner's choice.

The explanation that warm-up breaks

The warm-up letters are fixed across training cycles. Their oracle surprisal is zero, and the trained model predicts them well. On the canonical baseline, target-aligned cross-entropy averages 0.113 bits during warm-up, versus 0.766 bits during repetition. Yet warm-up activity is high. This contradicts the simple explanation that the activity trace follows model uncertainty everywhere.

There is a bookkeeping complication worth making explicit. Token zero attends to no earlier token under the BDH mask, producing exactly zero measured activity. Historical Python summaries included it: warm-up activity was 18.14% across 13 positions. The visible summary excludes this structural zero and becomes 19.65% across 12 positions. Memorization and repetition are unchanged. Cross-entropy uses its own predictor-to-target alignment; we do not discard loss entries merely because an activity convention discards token zero.

A possible explanation is that a cold sequence requires the model to establish state before later repetitions. That remains a hypothesis. Herrmann, Csordas, and Schmidhuber study hidden-state prediction as a measure of computation and explain why next-token loss can be inadequate. Their work motivates care about proxies; it does not establish our particular cold-start account. Measuring In-Context Computation Complexity via Hidden State Prediction

What a browser can honestly show

The page loads locally exported weights and computes a full causal forward pass in JavaScript. Activity and per-token cross-entropy are LIVE. The letters are SYNTHETIC. Oracle surprisal is ANALYTIC under the generator: new random letters have log2(26), approximately 4.70 bits, while known warm-up and repeated letters have zero. Those ideal values describe a generator with known structure, not an empirical claim about what the model knows.

The neuron grid shows actual computed coordinates for the selected token. Playback animates inspection of a completed computation; it is not fresh inference at every animation frame. That distinction lets us provide an engaging interface without turning animation into evidence. Scrubbing tokens should only change the view into cached activations. Changing the word or weight set requires new computation. The live Transformer adds a dashed activity curve and a second independently measured grid, synchronized by token and layer. Cell position does not imply that neurons correspond across architectures.

A JavaScript port needs more than a visually similar chart. The parity checks use Python reference tensors for trained and random weights, including every layer and token. Exact integer active counts and zero patterns are especially important because a tiny numerical difference near zero can change the measured event. Numerical tolerances check activation values, while logits and cross-entropy provide additional output checks. Both Transformer weight sets also pass their own parity suite. Passing these tests supports implementation agreement, not scientific generalization. A fresh-clone retraining on the current macOS ARM environment yielded about 3.37x at BDH layer 2, versus the preserved 3.10x. That variability is reported rather than replacing the shipped evidence; reproducible fixture inference and bit-identical optimization are different promises.

Comparing without moving the goalposts

The structural comparison uses the ReLU hidden activation of a Transformer and the gated product of BDH. The original Transformer FFN contains an output projection and biases; a compact hidden-activation formula must be labelled as a simplification. ReLU offers an informative exact-zero measurement here. GELU's nonzero fraction is defined, but generally does not serve the same purpose.

The separate offline Transformer control uses a 1,024-coordinate hidden layer and shared weights, but has 71,680 parameters. Equal denominators therefore do not mean equal parameter budgets. Its causal attention and positional dimensions also differ. PRECOMPUTED results belong to their fixed evaluation preset and must never appear to react to live word controls. The sole 2,200-step run passed its predeclared predictive-comparability gate. At layer 2, activity fell from 1.98% to 1.25%, a 1.58x ratio; evaluation cross-entropy was 0.638 bits. Thus this Transformer also becomes sparser on repetition. Its lower absolute fractions and different behavior across layers make a single ratio an inadequate architecture ranking. No retry or ratio-based tuning was performed.

This comparison is not Spark Transformer. You and colleagues explicitly introduce top-k masking in FFN and attention, illustrating one way to engineer sparsity. It would be wrong to use that example to claim every Transformer has a fixed activity budget. Our dense browser execution also does not turn a lower neuron count into a demonstrated speedup. Spark Transformer: Reactivating Sparsity in FFN and Attention

The next experiment, and the boundary of this one

A useful next experiment would compare the first cycle of a cold sequence against later cycles under the same trained model. We would predeclare phase windows, keep words and target alignment matched, and report both activity and cross-entropy. If warm-up activity falls only after context has been established, that would support the state-establishment hypothesis. If it stays high, the hypothesis would need revision. Neither outcome has been measured here.

We deliberately keep this artifact a synthetic instrument. It does not demonstrate natural-language transfer, semantic synapses, browser training, or energy savings. BDH-CQ is a later recurrent latent reasoning system, and our probe neither implements nor evaluates it. BDH-CQ: In-Context Learning with Recurrent Latent Reasoning

Not an official BDH model. The local implementation follows the public BDH architecture and retains its MIT notice. Earlier project records credit Claude assistance; this build uses Codex and parallel helper agents for implementation, documentation, and checks. The editable sources and recorded tests make those contributions reviewable. The strongest version of this demo keeps the successful curve and its counterexamples equally easy to find.